

TP 04

Programación II

Contenidos
- Implementación del TDA Stack - Implementación del TDA Queue

Consigna:

- Crear nuevos packages de nombre **stackModule** y **queueModule**, las interfaces **SimpleStack<E>**, **SimpleQueue<E>**, y las clases **SimpleArrayStack<E>**, **SimpleLinkedStack<E>**, **StackExercise**, **SimpleArrayQueue<E>**, **SimpleLinkedQueue<E>**, y **QueueExercise**, en los packages correspondientes.
- SimpleStack<E>**:
 - Métodos:**
 - `public void push(E element).`
 - Inserta `element` al final de la pila.
 - `public E pop()`
 - Remueve el último elemento de la pila y lo devuelve.
 - `public E peek ()`
 - Devuelve el último elemento de la pila sin removerlo.
 - `public void clear().`
 - Borra todos los elementos de la pila, dejándola vacía.
 - `public int size().`
 - Devuelve la cantidad de elementos en la pila.
 - `public boolean isEmpty().`
- SimpleQueue<E>**:
 - Métodos:**
 - `public void enqueue(E element).`
 - Inserta `element` al final de la cola.
 - `public E dequeue()`
 - Remueve el primer elemento de la cola y lo devuelve.
 - `public E peek ()`
 - Devuelve el primer elemento de la cola sin removerlo.
 - `public void clear().`
 - Borra todos los elementos de la cola, dejándola vacía.
 - `public int size().`
 - Devuelve la cantidad de elementos en la cola.
 - `public boolean isEmpty().`
- StackExercise y QueueExercise:**
 - Heredan de `Exercise`
 - Funcionan igual que los otros ejercicios
 - Dan la bienvenida la primera vez.
 - Muestran el estado de la estructura (sólo `isEmpty` y `size`) las siguientes.
 - Permiten repetir las operaciones repetibles sin volver al menú principal.
 - Funciones para `push/enqueue`, `pop/dequeue`, `peek` y `clear` de su estructura correspondiente.
 - `Pop`, `Dequeue` y `Peek` deberían chequear que la estructura no esté vacía antes.
 - `Peek` debería volver al menú principal, ya que repetirla muestra el mismo elemento.
 - `Clear` no debería llamar a `clear` de la estructura si está vacía.